In [3]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error,r2_score
from sklearn.preprocessing import StandardScaler
```

In [4]:
```
from sklearn.datasets import fetch_california_housing
#Load the housing dataset
housing = fetch_california_housing()
df = pd.DataFrame(housing.data, columns=housing.feature_names)

#Add the target variable (house prices) to the dataFrame
df['MedHouseVal']=housing.target

#display the first few rows of the dataset
print(df.head(20))
```

```
    MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  Latitude  \
0   8.3252      41.0  6.984127   1.023810       322.0  2.555556     37.88
1   8.3014      21.0  6.238137   0.971880      2401.0  2.109842     37.86
2   7.2574      52.0  8.288136   1.073446       496.0  2.802260     37.85
3   5.6431      52.0  5.817352   1.073059       558.0  2.547945     37.85
4   3.8462      52.0  6.281853   1.081081       565.0  2.181467     37.85
5   4.0368      52.0  4.761658   1.103627       413.0  2.139896     37.85
6   3.6591      52.0  4.931907   0.951362      1094.0  2.128405     37.84
7   3.1200      52.0  4.797527   1.061824      1157.0  1.788253     37.84
8   2.0804      42.0  4.294118   1.117647      1206.0  2.026891     37.84
9   3.6912      52.0  4.970588   0.990196      1551.0  2.172269     37.84
10  3.2031      52.0  5.477612   1.079602       910.0  2.263682     37.85
11  3.2705      52.0  4.772480   1.024523      1504.0  2.049046     37.85
12  3.0750      52.0  5.322650   1.012821      1098.0  2.346154     37.85
13  2.6736      52.0  4.000000   1.097701       345.0  1.982759     37.84
14  1.9167      52.0  4.262903   1.009677      1212.0  1.954839     37.85
15  2.1250      50.0  4.242424   1.071970       697.0  2.640152     37.85
16  2.7750      52.0  5.939577   1.048338       793.0  2.395770     37.85
17  2.1202      52.0  4.052805   0.966997       648.0  2.138614     37.85
18  1.9911      50.0  5.343675   1.085919       990.0  2.362768     37.84
19  2.6033      52.0  5.465455   1.083636       690.0  2.509091     37.84

    Longitude  MedHouseVal
0     -122.23        4.526
1     -122.22        3.585
2     -122.24        3.521
3     -122.25        3.413
4     -122.25        3.422
5     -122.25        2.697
6     -122.25        2.992
7     -122.25        2.414
8     -122.26        2.267
9     -122.25        2.611
10    -122.26        2.815
11    -122.26        2.418
12    -122.26        2.135
13    -122.26        1.913
14    -122.26        1.592
15    -122.26        1.400
16    -122.27        1.525
17    -122.27        1.555
18    -122.26        1.587
19    -122.27        1.629
```

In [ ]:

In [5]:
```
#Define the feature (independent variable) and target(dependent variable)
X=df[['AveRooms']]
y=df[['MedHouseVal']]

#Split the data into training and testing sets
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_state=42)

print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"y_test shape: {y_test.shape}")
```

```
X_train shape: (16512, 1)
X_test shape: (4128, 1)
y_train shape: (16512, 1)
y_test shape: (4128, 1)
```

In [6]:
```
#preprocessing
from sklearn.preprocessing import StandardScaler
scalar= StandardScaler()
scalar.fit(X_train)
X_train=scalar.transform(X_train)
X_test=scalar.transform(X_test)
print("Mean/std used for scaling:",scalar.mean_,scalar.scale_)
```

```
Mean/std used for scaling: [5.43523502] [2.38730258]
```

In [7]:
```
#Create a Linear Regression Model
model = LinearRegression()

#Train the model on the training data
model.fit(X_train,y_train)

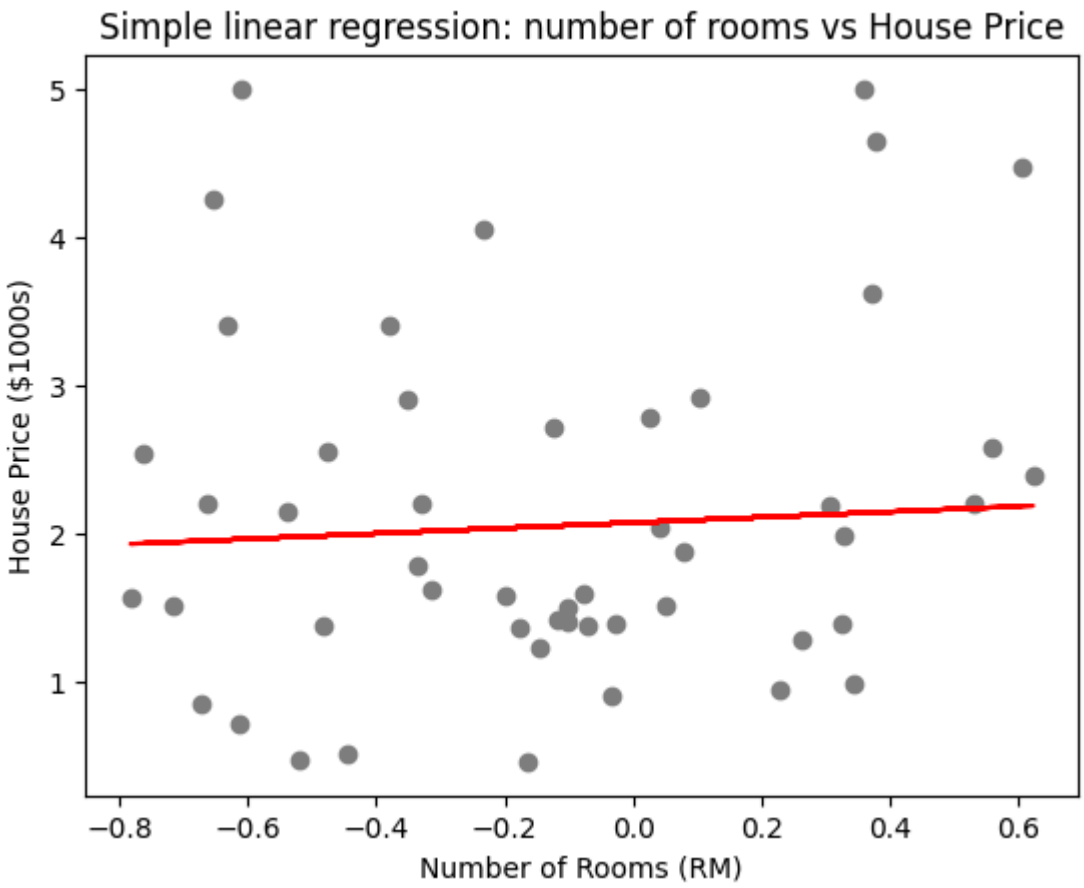
#print the intercept and coefficient
print(f"Intercept: {model.intercept_}")
print(f"coefficient: {model.coef_}")
```

```
Intercept: [2.07194694]
coefficient: [[0.18323882]]
```

In [13]:
```
y_predict_test = model.predict(X_test)
#predictions = pd.DataFrame({'Actual': y_test, 'Predicted':y_predict_test})
#print(predictions.head())
```

In [14]:
```
plt.scatter(X_test[:50],y_test[:50],color = 'gray')
plt.plot(X_test[:50], y_predict_test[:50], color='red')

#Add labels and title
plt.xlabel('Number of Rooms (RM)')
plt.ylabel('House Price ($1000s)')
plt.title('Simple linear regression: number of rooms vs House Price')
plt.show()
```



In [15]:
```
mse = mean_squared_error(y_test, y_predict_test)
print(f"Mean square error: {mse}")

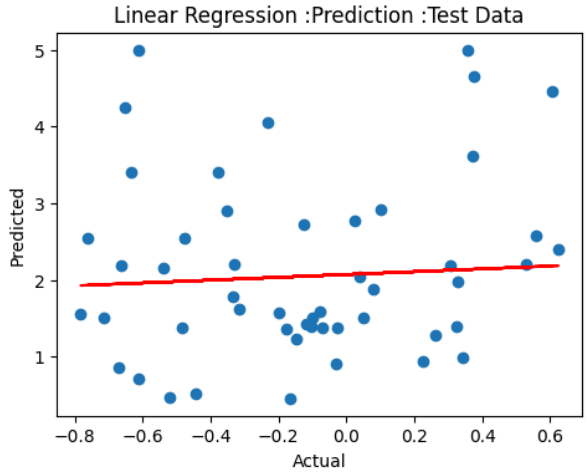
r2= r2_score(y_test,y_predict_test)
print(f"R-Squared score: {r2}")
```

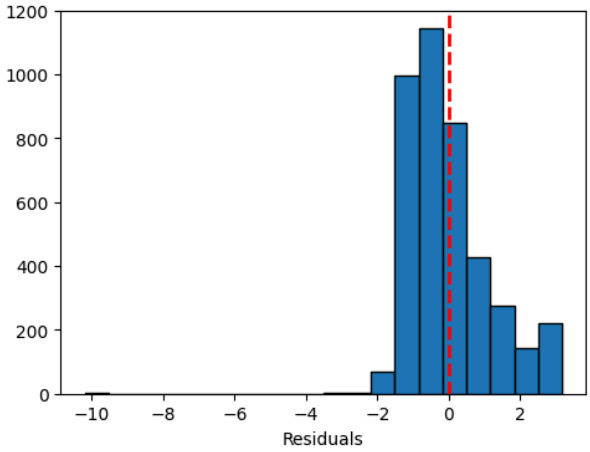
```
Mean square error: 1.29233144408073
R-Squared score: 0.01379533753228468
```

In [19]:
```
#two plots together and size of figure is 12x4
fig,axes = plt.subplots(1,2,figsize=(12,4))
axes[0].scatter(X_test[:50], y_test[:50])
axes[0].plot(X_test[:50], y_predict_test[:50], color='red',label='Regression')
axes[0].set_xlabel('Actual')
axes[0].set_ylabel('Predicted')
axes[0].set_title('Linear Regression :Prediction :Test Data')

#ploting the residual(error)
residuals=y_test-y_predict_test
axes[1].hist(residuals,bins=20,edgecolor='black')
axes[1].axvline(x=0,color='red',linestyle='dashed', linewidth=2)
axes[1].set_xlabel('Residuals')
```

Out[19]:
```
Text(0.5, 0, 'Residuals')
```


Linear Regression :Prediction :Test Data
Predicted
Actual
5
4
3
2
1
−0.8
−0.6
−0.4
−0.2
0.0
0.2
0.4
0.6


1200
1000
800
600
400
200
0
−10
−8
−6
−4
−2
0
2
Residuals

In [ ]: